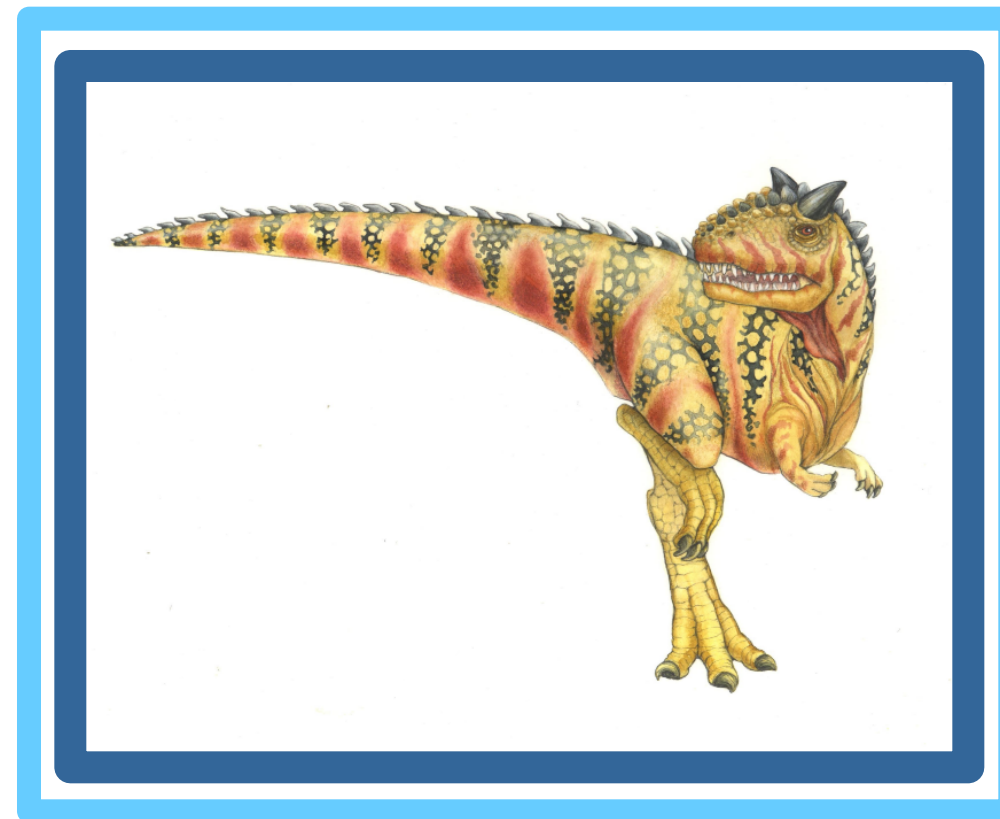
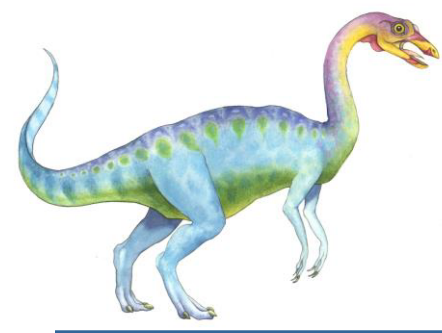


Chapter 8: Deadlocks





Chapter 8: Deadlocks

- ☐ System Model
- ☐ Deadlock Characterization
- ☐ Methods for Handling Deadlocks
- ☐ Deadlock Prevention
- ☐ Deadlock Avoidance
- ☐ Deadlock Detection
- ☐ Recovery from Deadlock





System Model

- System consists of resources
- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **request**
 - **use**
 - **release**





Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

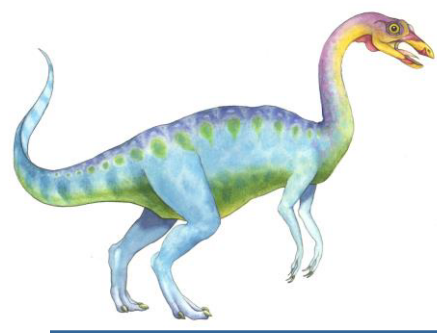




Methods for Handling Deadlocks

- Ensure that the system will *never* enter a deadlock state:
 - Deadlock prevention
 - Deadlock avoidance
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX



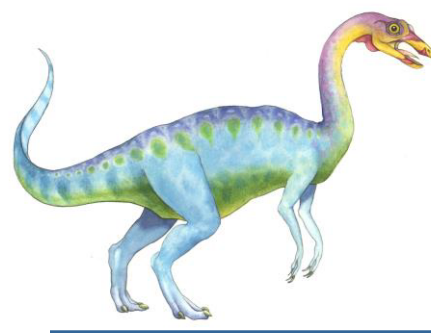


Deadlock Avoidance

Requires that the system has some additional *a priori* information available

- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes
- Two types of states - safe and unsafe

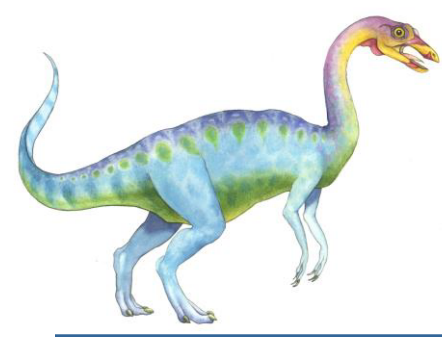




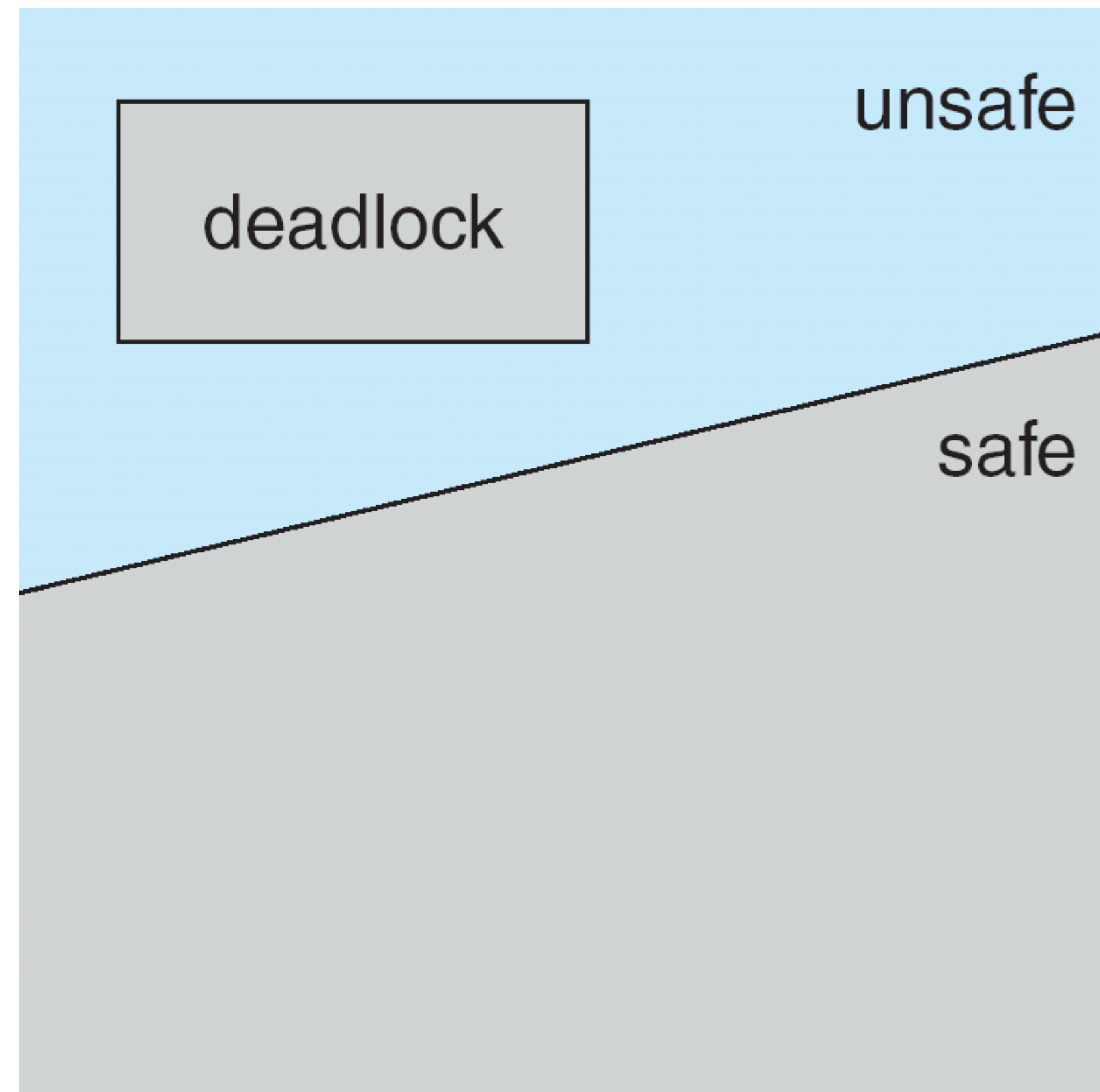
Safe State

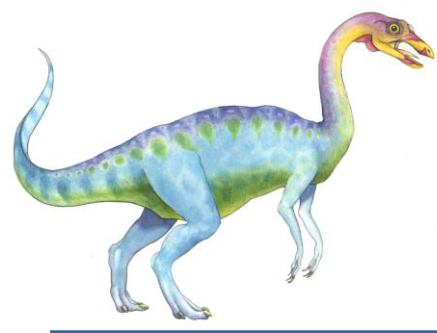
- When a process requests an available resource, system must decide if immediate allocation leaves the system in a safe state
- It will check Does the granting resources process can lead to deadlock
- System is in **safe state** if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ of ALL the processes in the systems such that for each P_i , the resources that P_i can still request can be satisfied by currently available resources + resources held by all the P_j , with $j < i$
- That is:
 - If P_i resource needs are not immediately available, then P_i can wait until all P_j have finished
 - When P_j is finished, P_i can obtain needed resources, execute, return allocated resources, and terminate
 - When P_i terminates, P_{i+1} can obtain its needed resources, and so on





Safe, Unsafe, Deadlock State

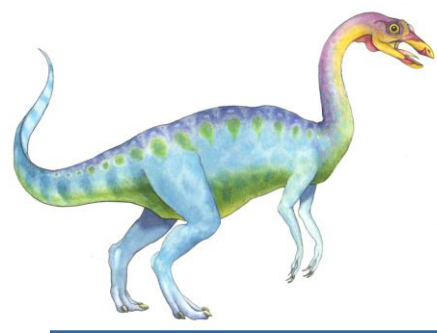




Basic Facts

- If a system is in safe state $\Rightarrow$ no deadlocks
- If a system is in unsafe state $\Rightarrow$ possibility of deadlock
- Avoidance $\Rightarrow$ ensure that a system will never enter an unsafe state.

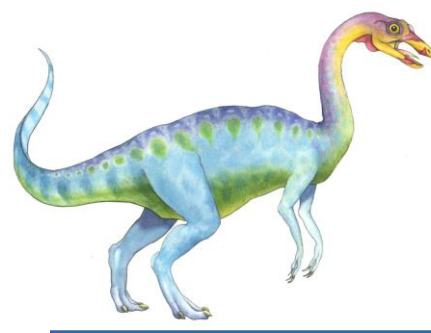




Avoidance Algorithms

- Single instance of a resource type
 - Use a resource-allocation graph
- Multiple instances of a resource type
 - Use the banker's algorithm

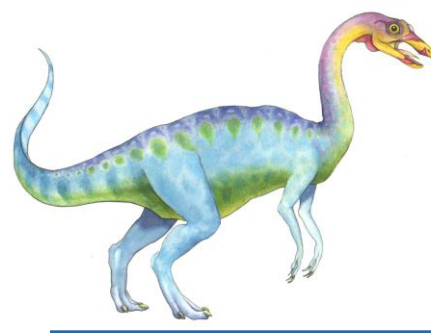




Banker's Algorithm

- ☐ Multiple instances
- ☐ We need prior information to avoid deadlock
- ☐ What kind of information we are looking for:
 - ☐ What is the max number of instances of a resource can a process request?
 - ☐ How many instances of a resource each process currently hold?
 - ☐ How many instances are available in the system?
- ☐ When a process requests a resource it may have to wait
- ☐ When a process gets all its resources it must return them in a finite amount of time



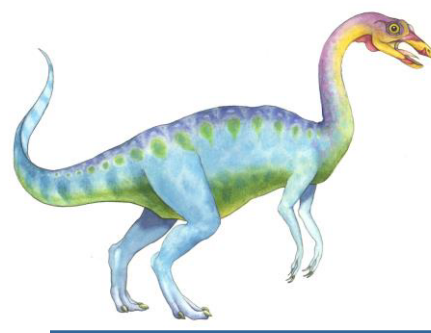


Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;
3 resource types:
A (10 instances), B (5 instances), and C (7 instances)
- Snapshot at time T_0 : (Max - total allocated resources for each processes)

	<u>Allocation</u>			<u>Max</u>			<u>Available</u>		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			





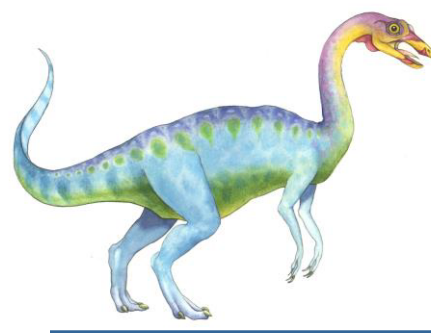
Example (Cont.)

- The content of the matrix **Need** is defined to be **Max – Allocation**

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- Use need matrix and compare it with availability and find the safe sequence.
- Create the safe sequence for p_0, p_1, p_2, p_3 and p_4
- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria





Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i, j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i, j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i, j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i, j] = Max[i, j] - Allocation[i, j]$$



Activity

Consider the following snapshot of a system in which four resources A, B, C and D are available. The system contains a total of 6 instances of A, 4 of resource B, 4 of resource C, 2 resource D.

	<i>Allocation</i>				<i>Max</i>			
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
P_0	2	0	1	1	3	2	1	1
P_1	1	1	0	0	1	2	0	2
P_2	1	0	1	0	3	2	1	0
P_3	0	1	0	1	2	1	0	1

Is the system in a safe state? Why or why not?

Is the system deadlocked? Why or why not?

Need and available Matrix

	<i>Allocation</i>				<i>Max</i>				<i>Need</i>				<i>Available</i>			
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
<i>P</i> ₀	2	0	1	1	3	2	1	1	1	2	0	0	6	4	4	2
<i>P</i> ₁	1	1	0	0	1	2	0	2	0	1	0	2				
<i>P</i> ₂	1	0	1	0	3	2	1	0	2	2	0	0				
<i>P</i> ₃	0	1	0	1	2	1	0	1	2	0	0	0				

Because P_0 's $Need = [1, 2, 0, 0] \leq Available = [6, 4, 4, 2]$, P_0 can run and return its $Allocation = [2, 0, 1, 1]$ to $Available$. The new $Available = [6, 4, 4, 2] + [2, 0, 1, 1] = [8, 4, 5, 3]$:

	<i>Allocation</i>				<i>Max</i>				<i>Need</i>				<i>Available</i>			
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
P_0					3	2	1	1					8	4	5	3
P_1	1	1	0	0	1	2	0	2	0	1	0	2				
P_2	1	0	1	0	3	2	1	0	2	2	0	0				
P_3	0	1	0	1	2	1	0	1	2	0	0	0				

Now we run P_1 . After that P_1 returns its $Allocation = [1, 1, 0, 0]$ to $Available$, and, hence $Available = \text{old } Available + P_1's \text{ Allocation} = [8, 4, 5, 3] + [1, 1, 0, 0] = [9, 5, 5, 3]$.

	<i>Allocation</i>				<i>Max</i>				<i>Need</i>				<i>Available</i>			
	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>
P_0					3	2	1	1					9	5	5	3
P_1					1	2	0	2								
P_2	1	0	1	0	3	2	1	0	2	2	0	0				
P_3	0	1	0	1	2	1	0	1	2	0	0	0				

Now we can run P_2 followed by P_3 . Consequently, the system is in a safe state. Because the deadlock state is in the unsafe area and because this system is in a safe state, this system is not deadlocked.

Safe Sequence — P0, P1, P2, P3

Quiz

1. What are the 4 conditions associated with deadlock?
2. What are the 4 approaches to handle deadlocks?
3. What are the two algorithms used for deadlock avoidance?
4. Which deadlock avoidance algorithm will be used when we have multiple instances?
5. How can we calculate the need matrix?
6. If there is a safe sequence, there is no deadlock and the system is in safe state — T or F?

In Next Class

Banker's Algorithm — Resource Request
Resource allocation graph based algorithms



Chapter 8: Deadlocks

System Model
Deadlock Characterization
Methods for Handling Deadlocks
Deadlock Prevention
Deadlock Avoidance
Deadlock Detection
Recovery from Deadlock

